

Explanation of PennySortingParallel program

1. Arrays.parallelSort() → Data Parallelism (built-in, optimized)

```
Arrays.parallelSort(arr1);
```

What it does

- Splits the array into **chunks**
- Sorts chunks **in parallel**
- Merges results back together

How it works internally

- Uses the **Fork/Join framework under the hood**
- Implements a **parallel merge sort / dual-pivot quicksort hybrid**
- Automatically decides when to switch to sequential sorting (threshold)

Key idea: Data Parallelism

- Same operation (**sorting**) applied to **different parts of the data simultaneously**

Why it's powerful

- Highly optimized by the Java runtime
- No manual thread management
- Usually the **fastest and easiest option**

Takeaway

“Let Java handle parallelism for you.”

2. Arrays.stream().parallel().sorted() → Functional Parallelism (Stream API)

```
int[] sortedArr2 = Arrays.stream(arr2)
    .parallel()
    .sorted()
    .toArray();
```

What it does

- Converts array → stream
- Marks stream as **parallel**
- Performs sorting across multiple threads

How it works

- Uses the **common ForkJoinPool**
- Breaks data into chunks behind the scenes
- Applies operations in a **pipeline**

Key idea: Declarative / Functional Parallelism

- You describe *what* to do, not *how*
- Java decides how to parallelize

Important nuance

- sorted() is a **stateful operation**
- Requires coordination across threads → can add overhead

Why it may be slower

- Stream setup overhead
- Boxing/unboxing (if not careful)
- Less control than parallelSort

Takeaway

“Parallelism expressed as a pipeline of operations.”

3. ForkJoinPool + ParallelMergeSort → Task Parallelism (manual control)

```
pool.invoke(new ParallelMergeSort(arr3, 0, arr3.length - 1));
```

What it does

- You explicitly define a **parallel merge sort**
- Breaks problem into **recursive tasks**

How it works

Your ParallelMergeSort likely:

1. Splits array into halves
2. Recursively sorts left + right in parallel
3. Merges results

Using:

- RecursiveAction
- invokeAll(leftTask, rightTask)

Key idea: Task Parallelism

- Different **tasks** run in parallel (not just data chunks)
- You control:
 - splitting
 - thresholds
 - merging

Why it's important

- Shows **how parallelism actually works under the hood**
- More flexible, but more complex

Downsides

- More code
- Easy to make inefficient (bad thresholds, merging overhead)

Takeaway

“You are the scheduler.”